Does a faster AI server mean a shorter wait for everyone?

A plain-English tour of an experiment on batching, caching, and chunking in LLM serving

Why I got curious

Picture a busy AI assistant. One person types a quick question. Right behind them, someone pastes in a 50-page document. Both requests land on the same GPU. Who waits, and for how long?

When people say a server is “faster,” they usually mean it produces more text per second overall. But that is not what you feel as a user. What you feel is the pause before the first word appears. It is possible to boost the first number while making the second one worse for some people.

So I set out to test it. Three well-known tricks are meant to make serving more efficient, and I wanted to know which ones help real users, when they help, and whether any of them have a catch.

The three tricks, in everyday terms

1. Batching: cooking several orders at once. A kitchen that cooks one meal at a time leaves the stove mostly idle. Cooking many orders together keeps everything busy. A GPU works the same way when it handles many requests simultaneously.

2. Prefix caching: remembering what you already read. If every request starts with the same long instruction (say, a company policy), why re-read it each time? Caching keeps the work already done on that shared beginning, so later requests skip ahead.

3. Chunked prefill: reading a long document in bites. Before answering, the model has to “read” the whole prompt. A huge prompt can hog the GPU. Reading it in smaller chunks lets other people’s short requests squeeze in between the bites. In theory that should be fairer.

How I tested it

I used a small open model (Qwen2.5-1.5B-Instruct) served with vLLM on a single NVIDIA RTX 4090. Everything stayed fixed except the thing I was testing: same model, same hardware, and every request asked for exactly 128 tokens of output, so runs are comparable.

I made up three kinds of traffic:

- **Unique prompts:** every request starts differently, so there is nothing to reuse.
- **Shared policy:** every request starts with the same ~2,000-token policy, then a unique task.
- **Mixed:** 75% short inputs and 25% long ones, like quick questions arriving alongside big documents.

Then I ran six server setups at three levels of demand (1, 8, and 32 users at once), three times each. I also reshuffled the order of setups and gave every one a fresh server, so one lucky or unlucky moment would not decide the result.

I tracked two things: **how much text came out per second** (throughput), and **how long until the first word showed up** (what users actually feel).

> **The scale:** 162 test cells, 20,736 timed requests, zero failures. The raw per-request records are published so anyone can recompute the numbers.

What I found

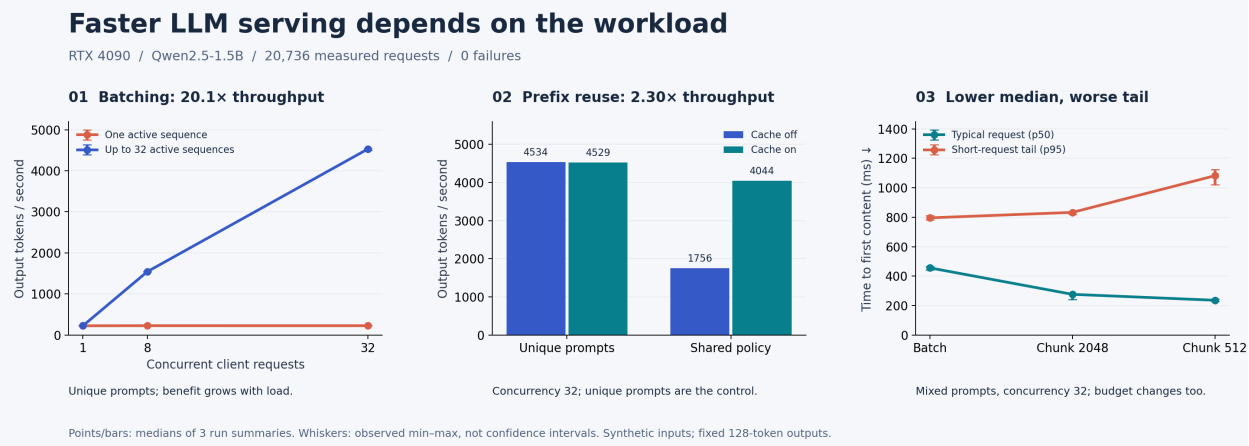


Figure: the three headline results, all measured at 32 simultaneous users.

Finding 1: batching is a huge win, once there is a crowd

With 32 users sending unique prompts, a server handling one request at a time managed about **226 tokens per second**. With batching it reached about **4,534**. That is roughly **20 times** more. The wait for the first word dropped from about 17.6 seconds to 0.22 seconds, because people were no longer standing in a single-file line.

The control matters, though: with just one user, both setups were equal (about 224 tokens/s). Batching only pays off when there is something to batch.

Finding 2: caching helps exactly when there is something to reuse

When all requests shared the same long policy, caching raised throughput from about 1,756 to 4,044 tokens per second (**2.3 times**) and cut the typical wait from 0.78 to 0.27 seconds.

When requests had nothing in common, caching changed nothing (4,534 vs 4,529 tokens/s). That is reassuring in two ways: it costs nothing when it cannot help, and the flat result tells us the gain really did come from reuse and not from some side effect.

Finding 3: a better typical wait can hide a worse unlucky wait

This was the surprise. On the mixed traffic at 32 users, I compared plain batching with chunked reading:

Setup	Tokens/s	Typical first-word wait	Slow-case wait for short requests (p95)
Plain batching	2,706	456 ms	796 ms
Chunks of 2,048	2,707	276 ms	833 ms
Chunks of 512	2,440	236 ms	1,082 ms

Each value is the median of three runs. “p95” means the wait that 95% of requests beat, so it describes the unlucky end.

Using 512-token chunks made the **typical** user happier: the median wait nearly halved (456 to 236 ms). But the short requests in the unlucky tail waited about **36% longer** (796 to 1,082 ms), and total throughput dropped by about **10%**.

Why might that be? My best guess is that cutting the big prompts into many small pieces creates a lot more turns to take, so short requests can end up waiting through more of them. I can’t confirm that from client-side timing alone; it is a hypothesis for future work.

It also depends on load. At 8 users, the same 512-token setup *improved* the short-request tail (257 to 212 ms). So “smaller chunks are fairer” is not a rule, only something that can be true at some loads.

> **Takeaway:** there is no single “faster.” Batching wins with a crowd, caching wins with shared beginnings, and chunking is a trade: better typical latency, possibly worse tail latency. Choose based on who you most want to protect.

How much should you trust this?

It is a careful but small study. Here is what to keep in mind:

- One small model on one GPU. Results may differ for bigger models or other hardware.
- Prompts are synthetic, and I measured speed only, not answer quality.
- Only three repetitions, and 128 requests per cell. That is enough to see direction, but not for very precise tail estimates.
- Users arrive in a steady closed loop here, whereas real traffic is burstier.
- The chunking comparison changes both the policy and the budget, so it shows a trade-off in setups rather than the effect of one single switch.
- The model version was not pinned, so an exact re-run might differ slightly.

The ranges I quote are just the lowest and highest of three runs, not formal confidence intervals.

What I would do next

- Repeat the mixed-traffic test with more requests and more independent sessions, to check the tail-latency penalty holds up.
- Use bursty, realistic arrivals instead of a steady loop.
- Sweep prompt lengths, how much prefix is shared, and chunk sizes to find where the trade-off flips.
- Record scheduler traces to explain *why* it happens, not just that it does.

Everything needed to check or re-run this work (code, 162 summaries, request records, file hashes) is in the project repository under `docs/` and `scripts/`. No GPU is needed to verify the published numbers.